

Discussion Unit: DSE Analytics

DSEFS

- The goal of this Discussion Unit is to configure and use DSEFS.

Discussion Lab:

Matching pairs – Match the attributes on the right with the areas on the left

DSE Analytics; DSEFS, HDFS, NAS,
RDD, Map/Reduce

DSE Analytics: HDFS, RDD, DSEFS, M/R, NAS



DSEFS



Map/
Reduce



SAN



Core to Spark



Core to Hadoop



POSIX compliant



Local to computation



Remote from computation



Schema on read



Schema on write

End of Discussion Lab:

- Staging for import/export
- Intermediate results in a DAG, Workflow
- Distributing Jars; save work for driver and executor
- Scan heavy workloads
- Unstructured data

- DSE 5.0+ (CFS, 2.0 to 5.1)
- HDFS compatible storage
- Compression (HW level encryption)
- Shared nothing
- Same JVM as DSE Core
- Secure, Local DSE, LDAP, Kerberos, also TLS/SSL
- Performant; option to disable fsync

DSEFS: What and Why

DSEFS



DSEFS: Features

- Creating, listing, moving, renaming, deleting files and directories, wildcard expansion
- Optional password / token authentication and Kerberos authentication in Spark
- POSIX permissions and file attributes
- Interactive console with access to DSEFS, local file system, and other HDFS-compatible filesystems
- Querying block locations for efficient data local processing in Spark or Hadoop
- Replication and block size control
- Transparent optional LZ4 compression
- Utility to check filesystem integrity (fsck)
- Utility to view status of the cluster (e.g. disk usage)

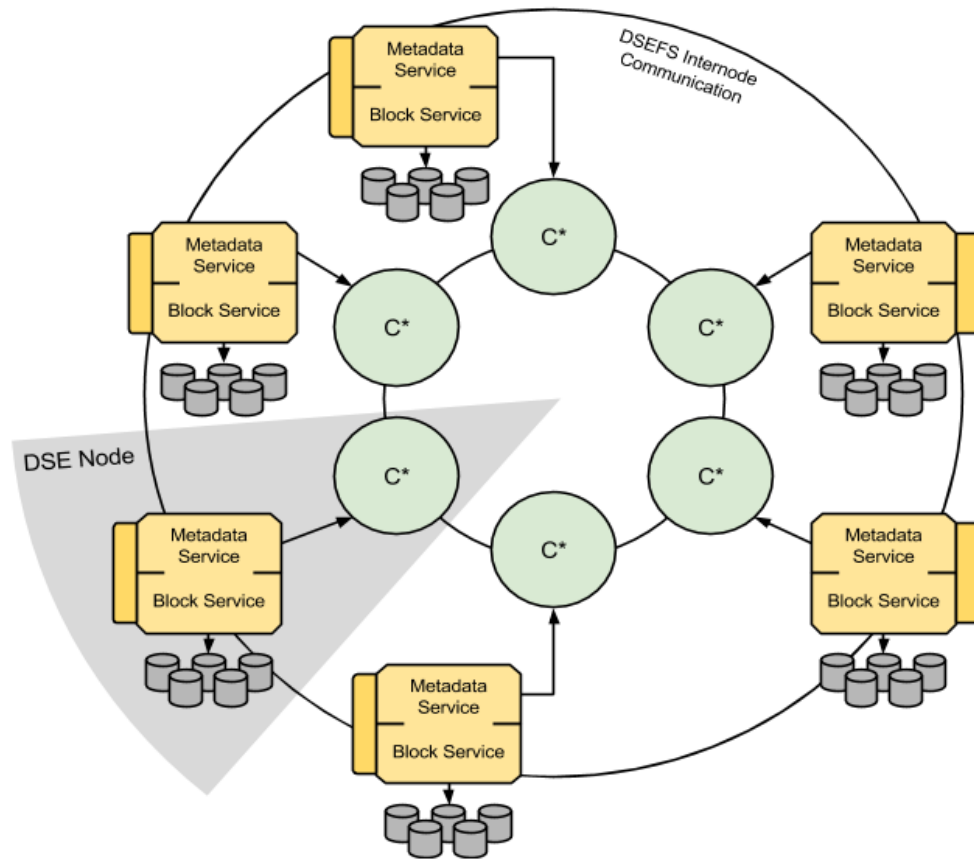
DSEFS: Just DSE Analytics (Apache Spark)



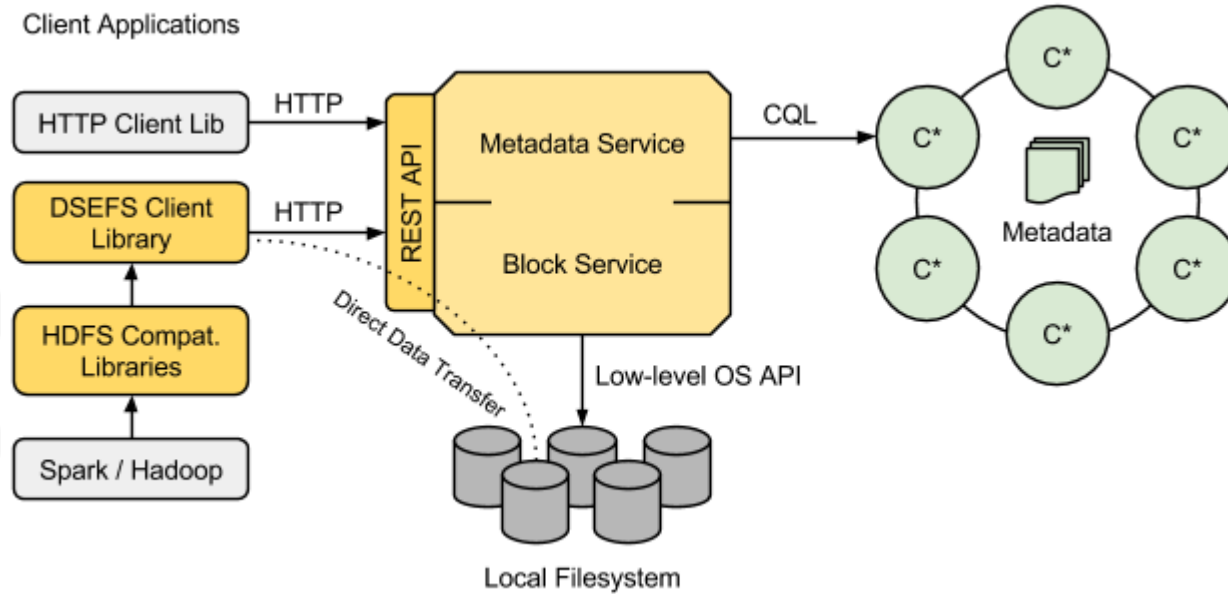
- Checkpointing (DAG)
- Distributing Jars; save work for driver and executor
- Intermediate results; (Parquet, good choice)
- Starts by default with DSE Analytics
- dse.yaml
- Prefer; separate disk-drive/controller
- Managed as Keyspace; RF and Replication Strategy
- Also RF per directory, per file
- One per DC. Multiple DCs ?, Different DSEFS names

DSEFS: Architecture

- No single point of failure; shared nothing
- Same JVM as DSE Core



DSEFS: Architecture



- Metadata is DSE Core
- Data-data as blocks on DSEFS

DSEFS: Roundtrip using DSE Spark



```
val rows = sc.textFile(  
    "file:///opt/stores_db/7545_HelloWorld.csv")  
rows.collect().foreach(println)  
  
rows.saveAsTextFile(  
    "dsefs:///tmp/7545_HelloWorld.csv")
```

DSEFS: Verify via "dse fs"

```
# dse fs
dsefs dsefs://127.0.0.1:5598/ > ls -l
Type Permission Owner Group Length Modified Name
dir   rwx-wx-wx  none  none      - 2018-07-19 17:09:49-0600 tmp
dsefs dsefs://127.0.0.1:5598/ > cd tmp
dsefs dsefs://127.0.0.1:5598/tmp/ > ls -l
Type Permission Owner Group Length Modified Name
dir   rwx-wx-wx  none  none      - 2018-07-19 17:09:49-0600 hive
```

... After job on previous page runs

```
dsefs dsefs://127.0.0.1:5598/tmp/ > ls -l
Type Permission Owner Group Length Modified Name
dir   rwxrwxrwx  none  none      - 2018-07-19 17:21:54-0600 7545_HelloWorld.csv
dir   rwx-wx-wx  none  none      - 2018-07-19 17:09:49-0600 hive
```



DSEFS: Verify via "dse fs"

```
dsefs dsefs://127.0.0.1:5598/tmp/ > cd 75*
dsefs dsefs://127.0.0.1:5598/tmp/7545_HelloWorld.csv/ > ls -l
```

Type	Permission	Owner	Group	Length	Modified	Name
file	rw-r--r--	none	none	0	2018-07-19 17:21:55-0600	_SUCCESS
file	rw-r--r--	none	none	48	2018-07-19 17:21:54-0600	part-00000
file	rw-r--r--	none	none	15	2018-07-19 17:21:55-0600	part-00001

```
dsefs dsefs://127.0.0.1:5598/tmp/7545_HelloWorld.csv/ > cat part*
```

111, Bob, Mary
222, Ted
333, Alice, Bob, Harold
444, Dave, Bob

```
dsefs dsefs://127.0.0.1:5598/tmp/7545_HelloWorld.csv/ >
```

DSEFS: Settings, dse.yaml

```
dsefs_options:  
  enabled:  
  keyspace_name: dsefs  
  work_dir: /var/lib/dsefs  
  public_port: 5598  
  private_port: 5599  
  data_directories:  
    - dir: /var/lib/dsefs/data  
      storage_weight: 1.0  
      min_free_space: 5368709120 # Change to 1024 for dev
```

" ... DSEFS nodes are either unavailable or low on free space ... "

All of DSE Analytics: Keyspace RF/Strategy

Different; 1 DSEFS per DC



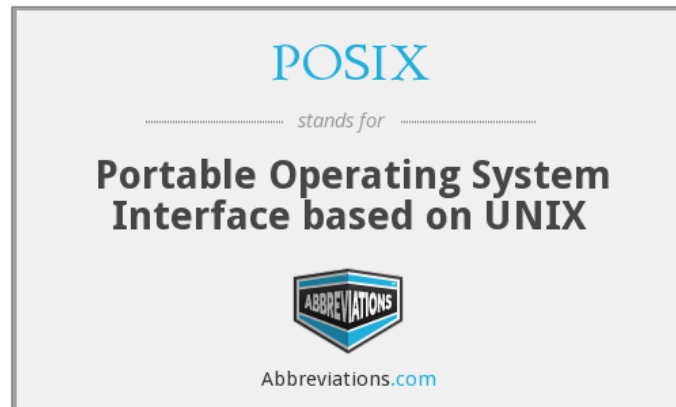
- dse_analytics
- dse_leases
- dsefs
- "HiveMetaStore"

- See DSE Core Practice Lab, 6207/Keyspaces

```
ALTER KEYSPACE dsefs  
WITH REPLICATION = {  
  'class': 'NetworkTopologyStrategy',  
  'Analytics': '3'};
```

- append
- cat
- cd
- chgrp, chmod, chown
- cp
- df
- echo
- exit
- fsck
- get
- ls
- mkdir, rmdir, rm [-r]
- mv, rename
- put
- pwd, realpath
- stat
- truncate
- umount

DSEFS: Nearly POSIX compliant



DSEFS: REST Interface

```
curl -L -X PUT 'localhost:5598/webhdfs/v1/fs/a/b/c/d/e?op=MKDIRS'  
curl -L -X PUT -T logfile.txt  
    '127.0.0.1:5598/webhdfs/v1/fs/log?op=CREATE&overwrite=\  
    true&blocksize=50000&rf=1'
```

```
curl -L -X POST logfile.txt \  
    'localhost:5598/webhdfs/v1/fs/log?op=APPEND'
```

Or from the DSE Spark shell:

```
val rdd1 = sc.textFile(  
    "webhdfs://localhost:5598/webhdfs/v1/fs/log")
```




End of Unit:



Additional Detail:

Using HDFS

Not part of the DSE system but is part of Hadoop

- Can be used for long term data storage
- Spark is able to connect to any existing HDFS
- Additionally Spark is designed to be file system independent
- Thus Spark can also connect to systems like HBase or S3

DSEFS Compared to: HDFS, CFS

Benefits of DSEFS over HDFS	Benefits of DSEFS over CFS
No single point of failure	No impact on DSE Core
No NameNode	No overhead from compactions, commit log writes
No SecondaryNameNode	No data density impact
No Zookeeper	Immediate deletes, no overhead

DSEFS Questions:

- What should the replication factor be on metadata?
- What should be the replication factor on a given file?
- What if the file is a jar needed to run a spark job (dse spark-submit -jars dsefs://myjob/jars/xyz.jar)?
- Where would you use DSEFS?
- Does DSEFS data count towards data density on a DSE node?
- Why use webhdfs?